

Self-Reflection and Failure Recovery in Agentic AI Coding Systems: A Systematic Review

Nakul Lagad^{1,*}, Ramirzaja Shethwala², Mohammad Arif³

¹M.Tech Student ²Assistant Professor ³Professor

Department of Artificial Intelligence and Data Science
Parul Institute of Engineering and Technology (PIET), Parul University
Vadodara-390019, Gujarat, India

E-mail: nakullagad084@gmail.com

E-mail: ramirzaja.shethwala17532@paruluniversity.ac.in

E-mail: arif_mohd2k@yahoo.com

*Corresponding author: Nakul Lagad

Abstract—Recent developments in Large Language Models (LLMs) have made it possible to design autonomous software engineering agents that can execute activities like code generation, repository management, debugging, testing, and planning. Yet, their reliability is constrained due to challenges in failure recovery, adaptation to unforeseen circumstances, and learning from their previous mistakes. In this paper, we propose a systematic literature review of 35 works on autonomous software engineering agents that consider the following aspects: self-reflection, memory, planning, repository-aware reasoning, cooperation between agents, benchmarking, and failure recovery. This work reveals several research gaps related to the lack of unified frameworks for failure recovery, root cause analysis, memory-assisted adaptive reasoning, failure recovery evaluation, and proactive failure avoidance. The results of the review underscore the importance of implementing reliable failure recovery techniques which would allow agents to recognize failures, adapt their actions, learn from their mistakes, and perform better in their execution.

Index Terms—Autonomous Software Engineering, Agentic AI, Large Language Models, AI Coding Agents, Self-Reflection, Failure Recovery, Repository-Aware Agents, Software Engineering Automation.

I. INTRODUCTION

AI has become increasingly prevalent in software engineering. The first tools based on AI have been aimed at completing code, spotting bugs, suggesting codes, and testing, while demanding significant human effort to make decisions and fix errors. Large Language Models (LLMs) have broadened this range of abilities by allowing systems to interpret natural-language requirements, produce and manipulate code and perform more complex software engineering tasks [1]–[3].

Nevertheless, software engineering in real life involves not only coding. Software engineering tasks may demand understanding of requirements, navigation through the repository, selection and manipulation of files, launching of development tools and tests, responding to results of execution [4]–[6]. All these requirements have led to the appearance of the agentic AI system capable of planning tasks, utilizing external

tools, performing actions, observing results, and changing its behavior in the direction of the desired goal [2], [3], [7], [8].

In software engineering, this trend has produced autonomous coding agents able to accomplish repository-based tasks with little human intervention. Systems including SWE-Agent, AutoCodeRover, MAGIS, AgentCoder, and Aider show the increasing capability of AI agents in checking repositories, changing source code, using tools, running tests, and iteratively refining the solution [5], [9]–[13]. However, despite these developments, autonomous coding agents suffer from susceptibility to failures caused by inappropriate planning, wrong interpretation of the context of a project, wrong selection of files, erroneous code generation, improper use of tools, and failed testing [9]–[11], [14]–[16].

One of the key limitations is that an agent can recognize the occurrence of an execution failure without being able to understand its cause, leading to inefficiency or repetition of repair attempts. To deal with this problem, various existing strategies have considered self-reflection, automated debugging, failure analysis, memory, planning, and failure recovery [15], [17]–[22]. Self-reflection allows an agent to review previous actions and give feedback for future actions, whereas failure recovery deals with diagnosing failures, changing the strategy of execution, and reaching the goal.

While advances have been made individually in each of these areas, there exists a lack of cohesion in the literature regarding reflection, memory, planning, debugging, and recovery. Specifically, the lack of a cohesive framework for recovery, root cause analysis, memory-based adaptive reasoning, and a thorough evaluation of the recovery process suggests that a synthesis of the current work is needed [15], [17]–[22].

In light of this, this review provides a systematic review of current literature in agentic AI coding systems, focusing specifically on self-reflection and failure recovery. This includes a discussion of autonomous coding agents, reflection processes, memory and planning methods, tool use, debugging and repair, and evaluation. This review also highlights existing

approaches, open problems, and future research directions for more reliable self-reflective coding agents.

The rest of this paper is organized as follows. Section II introduces the relevant background concepts relating to agentic coding systems and their associated failure-recovery mechanisms. Section III discusses the methodology of the review and the literature used. Section IV analyzes the current work and its advantages and disadvantages. Section V covers the primary research gaps, and Section VI covers the future research directions.

II. BACKGROUND

A. Agentic AI

An Agentic AI is an AI system that has the ability to pursue specific goals using reasoning, planning, decision making, manipulation of objects, memory and interaction with an environment [2], [3], [7], [8]. The difference between this kind of systems and traditional ones consists in the fact that while traditional systems only produce a response to a user's request, agentic systems act by cycles of interpretation of the current situation, choosing actions, observing their effect and changing future actions.

In software engineering, this allows an agent to shift from just generating code to executing a whole task by multiple steps. An agent can analyze a software problem, choose a sequence of actions, look at repository files, change file contents, run development tools, analyze the result and change the strategy if the outcome is not satisfy the task [3], [7], [16], [22]–[24].

B. Autonomous Coding Agents

Autonomous coding agents are agentic AI systems aimed at completing multi-step tasks of software engineering with minimal human intervention. Autonomous coding agents utilize LLM-based reasoning and perform the following steps: repository navigation, planning, tool usage, code modification, code execution, tests, and feedback [4], [9]–[12], [25]. This set of activities is distinct from conventional coding assistants that aid developers in code completion and suggestions of localized code changes.

Typically, the workflow of an autonomous coding agent starts with task interpretation followed by repository navigation and task planning. Next, the agent selects relevant files, makes modifications to the code, runs commands or tools of development, and assesses the resulting behavior using compiler output, tests, or other kinds of feedback from the environment. Subsequently, the results of the behavior can affect the actions of the agent [4]–[6], [11], [12], [25].

Some examples of systems that embody this paradigm include SWE-Agent, AutoCodeRover, RepairAgent, and OpenHands that use different sets of repository navigation, code repair, planning, and tool execution [4], [11], [12], [25]. The growing abilities of these systems have even led to the creation of repository-level benchmarks such as SWE-Bench for assessing the performance of systems on realistic software engineering tasks [5], [6], [23], [24].

However, there are still certain risks in using autonomous coding agents because of wrong task interpretation, wrong planning, mistakes in repository navigation, wrong file selection, wrong code generation, mistakes in tool execution, or failed tests. Since software engineering tasks involve inter-dependent execution steps, an error made during the initial stages may propagate to the later stages. Thus, self-assessment, failure diagnosis, and recovery become essential elements of autonomous coding [1], [3], [13], [15], [16].

C. Self-Reflection in Autonomous Coding Agents

The notion of self-reflection implies that an AI agent has the capability to examine its reasoning, actions, and outcomes, detect shortcomings, and utilize the feedback gained to make better decisions in future [13], [17]–[21]. As for autonomous coding, reflection gives another chance to assess the approach to solving the task in case something goes wrong with the generated code, test results, or output from some of the tools used.

There are several examples of such approaches. Reflexion makes use of verbal feedback from previous experiences to modify reasoning in the next attempt; SELF-REFINE distinguishes between generation, evaluation, and refinement of the solution; Teaching Large Language Models to Self-Debug uses reflective reasoning for debugging, and ExpeL makes agents capable of using the experience gained previously [13], [17]–[21].

Typically, the reflective cycle includes generation of a solution, its execution or evaluation, detection of undesirable outcomes, feedback generation, and modification of further solution. The reflective cycle is especially applicable for autonomous coding as navigation of a repository, file selection, the use of a tool or code generation can affect all other stages, from compilation, through testing to execution. Thus, reflection allows getting information about the necessity of change in the approach rather than repeating the same actions [1], [4], [11], [12], [15].

Reflection approaches typically concentrate on outcome evaluation after execution, not continuous reasoning analysis or cause detection. Such a drawback justifies the combination of reflection with memory and failure-recovery mechanisms [15], [22], [28].

D. Failure Recovery Mechanisms

Failure recovery involves an agent's capability to identify an execution failure, determine its cause, change its reasoning or execution strategy and then pursue the task to its successful completion [1], [3], [15], [16], [22]. It does not involve repeating the same failed action, but may include learning about the reason behind the action that led to the failure.

Autonomous coding may face failures at interpreting a task, planning the action, searching through repositories, selecting files to work with, code generation, tool usage, API invocation, compiling, testing and debugging [1], [4], [5], [11], [12], [15]. These actions depend on each other, therefore an initial failure

may result in further failures during execution. Recovery in this case requires both failure identification and adjustment.

The existing methods apply reflection, memory, planning and tool adaptation to enable failure recovery [1], [13], [17]–[19], [26], [27]. A common structure of a recovery loop includes failure detection based on execution logs, compiler messages, test results, runtime exceptions and tool output; failure cause identification; production of feedback; revision of an execution plan; and the execution of the revised plan. The loop is continued until the successful task completing or certain termination criteria are met [1], [6], [15], [22], [26], [27].

Failure recovery plays a key role in repository-based software engineering because agents should coordinate the activities of repository scanning, dependency analysis, source code modifying, compiling, testing, debugging and validation. Therefore, the recovery may need not just code fixing, but also changes in plans and tools as well as maintaining useful execution history between several iterations [1], [4], [11], [12], [20], [22], [27].

Nevertheless, the existing solutions still lack something. There are approaches that repeat a reasoning after a failure without properly determining its cause as well as those providing reflection but without saving previous experience. Also, most of them focus on individual execution failures and do not consider a need for continuous adaptation throughout the software engineering process [1], [3], [15], [22], [26], [27].

E. Memory Systems in Autonomous Coding Agents

Memory systems provide autonomous coding agents with the ability to save, retrieve, and re-use information gained from past experiences. Different from conventional LLM reasoning that is mainly driven by the present context, long-running software engineering tasks would need to retain information on past decisions, past commands, code modifications, debugging results, tool interactions, and execution failures [3], [7], [15], [27].

Broadly speaking, memory could be classified into short-term and long-term information retention. Short-term memory keeps the information needed for the current task, such as user inputs, execution logs, intermediate plans, tool outputs, and source-code changes. Long-term memory stores the useful experience such as past failures, debugging knowledge, repository information, good solutions, and past tool use experience that would be reused in future tasks [3], [15], [26], [27].

The recent memory systems have investigated various structured memory types, such as episodic memory, semantic memory, and procedural memory. Techniques such as A-MEM try to structure the valuable experiences and retrieve relevant information during future reasoning [15], [26], [27]. Memory thus would allow self-reflection and failure recovery by offering previous debugging experience, execution trace, patches, and failure recovery methods whenever similar failures would occur [1], [15], [22], [26], [27].

However, in spite of such progress, memory systems suffer from difficulty in determining what kind of information should

be saved, retrieved, updated, or removed. In addition, the retrieved memories could be irrelevant, outdated, or contradictory, and could thus influence future reasoning. Moreover, the current techniques fail to combine memory with reflection, execution monitoring, and failure analysis [3], [15], [26], [27].

F. Planning and Tool Utilization

The ability to plan allows for coding agents to implement high-level software engineering goals by breaking them down into a series of steps. Planning includes task decomposition, repository exploration, dependency analysis, code modification, testing, debugging, and validation, but allows the execution strategy to be changed in reaction to feedback from the environment [3], [5], [7], [16], [22].

The autonomous coding agents are dependent on external development tools due to the necessity to interact with environments that are outside of the language model. The tools could include terminals, repositories, compilers, debuggers, testing frameworks, version-control systems, documentation, and APIs to retrieve the necessary information, execute actions, and validate solutions [4], [5], [11], [12], [25].

Therefore, planning and tool usage make up a feedback loop where the agent chooses the appropriate actions to take, invokes the necessary tools, observes their output, and adjusts its future actions accordingly. Systems such as SWE-Agent, AutoCodeRover, RepairAgent, and OpenHands exhibit various combinations of planning, interaction with the repository, and using external tools [4], [11], [12], [25].

However, poorly implemented planning and/or poor choice of tools might cause new failures. The agents might be doing redundant actions, choosing incorrect tools, or failing to adapt their planning strategy when encountering unexpected execution results. Such flaws can lead to cascading failures during various phases of autonomous software engineering [1], [3], [5], [15], [22].

G. Evaluation Benchmarks for Autonomous Coding Agents

The rise of the complexity of autonomous coding agents requires benchmarks that would allow for evaluation of software engineering skills of an agent going beyond the capability of generating the required code. Repository tasks require the understanding of project structure, modifying source code, using development tools, executing tests, debugging and validating results iteratively [4], [14], [23], [25], [28].

Among the benchmarks there is SWE-Bench which is designed around the GitHub issues taken from open-source repositories. This benchmark asks the agent to understand the repository structure, find relevant source files, generate patches, execute tests and check whether the suggested solution fixes the problem [4], [21]. Some other benchmarks cover broader capabilities of agents. For example, AgentBench evaluates the performance of LLM-based agents in interactive environments including reasoning, planning, tool use and decision-making while API-Bank tests the ability to use external APIs and RedCode considers security in code generation and execution [23], [25], [28]. Another possible source of

evidence for studying AI coding assistants and autonomous coding agents is AIDev dataset where AI assistants take part in software development activities on a large scale like pull requests and repository actions [19].

While the existing benchmarks allow to evaluate task completion and performance of software engineering, they usually focus on the output rather than the process of reasoning, planning, memory usage, self-evaluation or recovering from failures. Consequently, the evaluation of autonomous coding agents as their capabilities grow must include the efficiency of reasoning, the adaptation of the agent after the failure and learning experience from the previous executions [1], [3], [15], [28].

III. REVIEW METHODOLOGY

This paper will examine recent works in relation to self-reflection and failure recovery mechanisms of autonomous artificial intelligence coding. Literature review was used as the methodology to analyze existing research in the fields of autonomous software engineering agents, reflective thinking, memory, planning, use of tools, and recovery methods. This review aims at providing an insight into the current state of affairs in the field while highlighting its limitations and opportunities for future research.

A. Literature Search Strategy

All the relevant publications have been identified through the most reputable scientific database, which include IEEE Xplore, ACM Digital Library, SpringerLink, ScienceDirect, Scopus, and arXiv. This is due to the fact that the above mentioned databases consist of high-quality journal papers, conference papers, and preprints related to the domain of artificial intelligence, software engineering, large language models, and autonomous agent systems.

The relevant literature was searched using keywords in different combinations including: "Agentic AI", "Autonomous Coding Agents", "Self-Reflection", "Failure Recovery", "LLM Agents", "Memory Systems", "Planning", "Tool Use", "Software Engineering Agents", "SWE-Agent", "OpenHands", "AutoCodeRover", "RepairAgent", "Reflexion", "SELF-REFINE", and "SWE-Bench". The Boolean operators such as AND and OR were used in the search procedure.

B. Inclusion Criteria

The chosen studies met one or more of the following criteria:

- Studies related to the use of autonomous AI coding agents and agentic software engineering.
- Research works that propose ideas for self-reflection, iterative development, or improvement of reasoning skills.
- Research studies dealing with recovery from failure, debugging, or autonomous software repair.
- Papers with proposals for memory systems, plans, and tools for autonomous software agents.
- Benchmark data sets for autonomous coding systems.

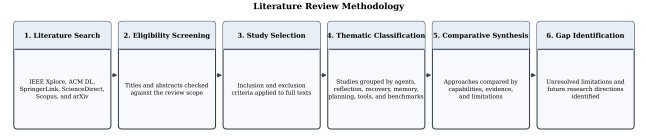


Fig. 1. Overview of the literature review methodology adopted in this study.

TABLE I
CLASSIFICATION OF THE SELECTED LITERATURE ACCORDING TO THE RESEARCH TOPICS

Research Topic	Main Focus of the Research Area	References
Autonomous Coding Agents	Exploring repositories, code writing, debugging, testing, and software engineering at the repository level	P01–P06
Self-Reflection	Reflective reasoning, iterative improvement, self-evaluation, and self-debugging	P07–P12
Failure Recovery	Detection of failure, failure diagnosis, recovery techniques, and repair of software automatically	P13–P17
Memory Mechanisms	Short-term memory, long-term memory, episodic memory, and memory retrieval	P18–P22
Planning and Tool Usage	Task decomposition, planning for execution, tool selection, command execution, and workflow modification	P23–P29
Evaluation Benchmarks	Benchmark datasets, evaluation frameworks, task-success measurement, and agent performance analysis	P30–P35

- Peer-reviewed articles, conference proceedings, and important preprints written during the period between 2023 and 2026.

C. Exclusion Criteria

The following studies were not included in the review:

- Papers irrelevant to autonomous software engineering and coding agents.
- Studies dedicated to the topic of code generation without autonomous reasoning and execution.
- Research papers with insufficient technical information or experiments.
- Duplicate papers and foreign language literature.
- Opinion pieces, blog entries, tutorials, and technical reports.

D. Paper Classification

After the process of screening was done, the articles were grouped together based on what they have contributed towards research. These groups include autonomous coding agents, self-reflection techniques, failure recovery methods, memory techniques, planning and tools usage, benchmarks, and evaluation. This categorization allows for an easy comparison of approaches that have been done previously and the identification of the trends, shortcomings, and gaps in the research field.

IV. LITERATURE REVIEW

As LLMs evolve rapidly, intelligent software engineering has moved beyond assisting coding into developing autonomous agents that can accomplish complex development tasks with minimal human involvement. Unlike the conventional code generation tools that mainly assist code completion and generation, the agentic approach to intelligent software engineering combines reasoning, planning, knowledge of

repositories, tool utilization, memory, and decision-making in order to aid multi-step software engineering tasks.

The literature that is currently available discusses many research areas such as repository-aware coding agents, multi-agent collaboration, self-reflection, iterative reasoning, memory-driven learning, debugging, failure recovery, tool utilization, planning, and benchmarking. Despite significant progress made in the autonomous software engineering field in these research areas, each of these research topics is usually studied separately.

In order to structure the review, the studies examined in this paper will be divided into five major thematic groups. The first group deals with the evolution of intelligent software engineering agents. The second one considers self-reflection and iterative reasoning, whereas the third one addresses memory-based learning and experience reusing. The fourth one deals with the problem of failure recovery and autonomous debugging. The last one addresses planning, tool use, benchmarking and new agent architectures.

This paper uses integrative analysis as a way of examining each line of research with respect to its evolution, abilities, weaknesses and interrelation with other components of autonomous software engineering. Such an approach enables to see the field in general and helps find gaps in failure recovery.

A. Evolution of Intelligent Software Engineering Agents

Intelligent software engineering agents have been developed alongside advances in LLMs, evolving from simple code assistance to full autonomy in software engineering. Initially, AI-based coding assistants offered only simple suggestions such as code snippets and completion based on context. Despite increasing the productivity of developers, they were still not able to function without human assistance when it came to repository exploration, reasoning, debugging, and managing software. Recent research focuses on fully autonomous agents capable of repository understanding, task decomposition, tool interaction, and execution feedback utilization.

The shift is not only motivated by advances in language model technology but also by reasoning, planning, understanding repositories, collaboration, and tool interaction in agentic systems. As shown in recent literature, LLM-based software engineering agents may assist in activities such as requirement analysis, implementation, testing, debugging, maintenance, and software evolution [1], [2].

Moreover, repository-level programming agents illustrate this trend. Unlike traditional code generators, they have the capability to explore project structure, find appropriate files, analyze dependencies, modify code, use development tools, and check the results by executing and testing generated programs. Examples of such agents include AutoCodeRover, SWE-agent, RepairAgent, and OpenHands [3]–[6].

Also worth noting is the trend of moving from assessment of stand-alone code quality to assessment of autonomous task execution. In recent literature, planning, actions sequence, tool usage, intermediate error handling, and successful completion of the repository-level task is being more and more taken into

account. Consequently, assessment is shifting from function-level correctness to more realistic software engineering scenarios.

Despite this progress, autonomous coding agents continue to be susceptible to errors that arise during the process of exploring repositories, planning, using tools, and intermediate reasoning. These errors could propagate over subsequent execution steps; however, many systems still depend on re-execution and/or repair without detecting the root cause of failure or learning from their failures.

Overall, the literature indicates an unmistakable shift from AI-based assistance in programming to more autonomous agents for software engineering that have greater capabilities in reasoning, planning, repository analysis, and tool usage. Nevertheless, robust long-horizon execution proves to be difficult enough to warrant studies of self-reflection, memory, and failure recovery. These topics are discussed in the subsequent subsections.

B. Self-Reflection and Iterative Reasoning in Autonomous Software Engineering Agents

In this regard, as autonomous software engineering agents continue to emerge, their success is determined not only by their capability of formulating proper solutions but also by their ability to evaluate, revise, and enhance their own reasoning processes in the course of task execution. While traditional code generation approaches tend to yield one solution for one question, autonomous agents run through long horizon tasks such as exploring repositories, task decomposition, invoking tools, code transformation, testing, debugging, and decision making. In this context, an erroneous choice of actions taken at the early stages of the process can have a cumulative effect and cause the agent to fail with the task at hand. In order to address this problem, more research has been done into self-reflection and iterative reasoning.

Reflection is regarded as one of the most effective paradigms for enhancing the reasoning skills of autonomous agents. In contrast to accepting their solution without any further deliberations, reflective agents critically review their reasoning at the intermediate stage, detect possible contradictions, provide feedback in order to correct them and finally change their way of decision making. Research on the impact of verbal reinforcement learning, self-feedback loop and reflective reasoning on the efficiency of task completion shows that reflecting on one's mistakes helps in performing the task better [7]–[9]. All the three paradigms mentioned above show that reflection acts as an internal supervisor helping autonomous agents to improve their reasoning skill without being trained.

On top of reflective reasoning, iterative reasoning has also emerged as an important area of study towards improving software engineering by means of autonomous software engineering. Rather than considering software production as one inference task, modern software engineering is being studied as a continuous optimization task where repeated reasoning leads to better and better solutions through reasoning. Self

TABLE II
EVOLUTION OF INTELLIGENT SOFTWARE ENGINEERING AGENTS

Research Stage	Primary Characteristics	Major Capabilities	Representative Studies	Remaining Challenges
AI-Assisted Programming	LLMs generate code suggestions and completion, but developers have full control of software development.	Code generation, code completion, code explanation, development assistance.	Early AI programming assistants	Limited reasoning abilities, non-autonomous, intensive human involvement.
Repository-aware Coding Agents	Agents understand repository architecture and can accomplish project-based software engineering activities.	Repository navigation, bug detection, code modification, testing, bug fixing.	AutoCodeRover [3], RepairAgent [5]	Agents lack the ability to adapt after errors during their execution.
Interactive Autonomous Agents	Agents interact with development environments via external tools and command line interface to accomplish software engineering tasks.	Tool invocation, terminal interaction, iteration, repository exploration.	SWE-agent [4], OpenHands [6]	Execution errors propagate to multiple reasoning steps.
Collaborative Software Engineering Agents	Multiple autonomous agents collaborate to accomplish difficult software engineering tasks by decomposing them and coordinating.	Planning, collaboration, task distribution, coordinated reasoning, workflow management.	Multi-Agent Software Engineering Survey [1], Software Engineering 3.0 [2].	Coordination complexity, scalability, evaluation, and communication overhead.
Towards Autonomous Engineering	Intelligent Software Agents Contemporary research centers on agents with high autonomy that have capabilities to reason, plan, learn and adapt during software engineering.	Autonomous reasoning, planning, tools usage, execution and software engineering automation.	Contemporary agentic software engineering research	Self-referencing, failure recovery, memory and autonomous decision-making still constitute an open research question.

debugging and iterative reasoning have shown that using execution feedback, agents can be guided to recognize any logical contradictions, fix the mistakes in implementation, and generate better and better solutions [8], [10]. Iterative reasoning improves robustness in comparison to one shot code generation.

Moreover, current research has even enhanced the scope of self-reflection not only by extending it beyond personal reasoning but also through experiential learning and collaborative intelligence. Instead of forgetting the execution history once the task is performed, modern autonomous agents maintain not only successful but also unsuccessful experience that could be used in future reasoning. Through experiential learning, agents are able to utilize their previous experience when facing the same software engineering problems, thus avoiding repeated mistakes in reasoning and making better decisions in the long term [11]. In addition, through reflective collaboration of several agents possessing specific reasoning capabilities, it becomes possible to assess the intermediate decisions, provide each other with feedback and perfect solutions collaboratively [12].

In addition to the increasing significance of reflective reasoning itself, this has led to a number of thorough studies on self-correction methods. The surveys available group the existing methods into iterative improvement, feedback-based correction, verification via external information sources, self-consistency reasoning, and adaptive revision techniques [13]. All of the studies taken together show that while the method of reflective reasoning always leads to increased quality of results, most existing approaches still rely on task-specific implementations and mainly aim at improving final results, not controlling reasoning during full work processes. The same observations have been made concerning the autonomous computer agents working in interactive software systems [14].

Even with all the advancements realized through the application of reflective reasoning technology, there are still a number of critical problems that exist. First, most of the existing models consider the process of self-reflection as something that occurs after execution rather than a continuous process of autonomous decision-making during execution. Second, the present models are more concerned with evaluating the results

of execution rather than the process of reasoning itself. This is because reflection will only realize that there has been some sort of mistake made but will not explain why the mistake has been made and how reasoning can be changed in the future to prevent such mistakes from happening.

In general, current research indicates that self-reflection and reasoning improvement considerably increase the efficiency of autonomous software engineering agents through their ability for continuous refinement of reasoning, learning, and improvement of execution efficiency. This method is critical for reliable autonomous software engineering. However, reflection does not suffice to ensure robust autonomous functioning as reasoning improvement presupposes that agents store information about past tasks. As a result, recent studies focus on memory-based learning that would supplement the process of reflective reasoning providing means for accumulation, structuring, and retrieval of information from long execution history. This memory system will be analyzed in the next section.

C. Memory-Driven Learning in Autonomous Software Engineering Agents

Whereas self-reflection makes it possible for software engineering agents to perform reasoning evaluation and improvement while performing a certain task, sustainable autonomous task performance calls for the capability to persist knowledge past execution instances. Software engineering tasks are repetitive in nature and are characterized by interactions with various artifacts including repositories, APIs, development tools and debugging environments. Thus, treating a software engineering task as a unique reasoning instance may lead to errors, redundancy and inefficient decision making. Recent research has, in this regard, focused on memory-based learning strategies that make it possible for autonomous agents to retain knowledge from different software engineering tasks.

Modern research findings indicate that memory structures are gradually shifting from being based on conversational history to becoming knowledge management systems. Modern agentic memories incorporate structures of information storage through semantic connections, hierarchies of memory, and methods of retrieval which are similar to those used

by humans. Unlike memory structures that store the entire conversation history, the contemporary memory structure only stores useful experience of execution while updating irrelevant information continually. Adaptive memory structure helps the autonomous agent to access task-related knowledge without flooding the reasoning process with unnecessary information [15], [16].

Another important area of research is concerned with long-term memory as a technique for continuous learning. Current literature indicates that having consistent memory across several interactions is very helpful when it comes to increasing the reasoning consistency of an agent, decreasing its execution errors, and improving its ability to complete long-term tasks. Instead of considering reasoning only with respect to the immediate execution context, long-term memory makes it possible to store gained knowledge for longer periods of time, which is useful for decision making during future software engineering processes. Evaluation results using long-term reasoning and conversation benchmarks also show that memory-augmented agents always outperform agents without memory [17].

Apart from information storage, recent studies on memory have come to regard memory as an active reasoning process rather than a passive information store. The recent memory systems are designed such that they are continuously interacting with the reasoning, planning, and action modules during the entire process of decision making. Memories retrieved from storage help in task decomposition, choice of tools, navigation through the repository, and debugging through the provision of solutions used before. Thus, memory not only helps in information storage but also makes the reasoning process more efficient and saves time and effort in exploring unproductive avenues.

Nevertheless, some critical challenges still have to be addressed in this regard. First of all, most current memory systems concentrate mainly on providing an efficient mechanism for accessing the information, without paying much attention to analyzing the quality of the memories themselves. Therefore, faulty memories will continue to affect the process of reasoning, causing wrong conclusions to be made. Secondly, current methods do not consider memory handling in conjunction with reflective reasoning and execution monitoring, which makes it impossible for the agent to make a decision about which experiences need to be saved, modified, or deleted after executing the task successfully or unsuccessfully. Finally, the technique of memory access still relies heavily on similarity searches that may lead to ignoring valuable experiences in new tasks.

Limitation number two refers to the poor integration of memory components and autonomous software engineering processes. Even though current architectures enable the retention of past experience, there is relatively little research on how traces of execution, debugging process, reflections, and failures can be integrated into one memory component in order to enhance future autonomous reasoning capabilities. As a result, the majority of intelligent software engineering

agents are still utilizing memory components that enable them to retrieve information, but not learn from past experience.

Thus, generally, the existing literature reveals the fact that memory-based learning has become a key element in the development of modern autonomous software engineering agents. With memory architecture allowing for knowledge persistence, adaptation, and learning, autonomous software engineering becomes more consistent in reasoning and decision-making process. However, at the same time, the existing memory methods mostly deal with increasing the availability of the information rather than solving the problems connected with failed execution processes. In order to make an agent autonomous and reliable, it is necessary not only to allow it to remember its past experiences but also to explain to it the reasons for failures, diagnose them, revise its reasoning, and restore itself from failed attempts. The issue will be covered in the next subsection.

The evolutionary trend that characterizes intelligent software engineering agents discovered during literature review is depicted in Fig. 2.

While contemporary agents utilize reasoning, tool interaction, self-reflection, and memory-based learning more than before, the area of resilient recovery remains relatively poorly developed. This issue will be considered in greater detail in the next section.

D. Failure Recovery and Autonomous Debugging

The increased autonomy of intelligent software engineering agents has greatly enhanced their capabilities to successfully perform complicated programming tasks with little to no human intervention. On the other hand, increased autonomy brings new problems of execution reliability, which means that while in the case of usual software programs the computer simply performs the set of actions that were programmed in advance, in the case of autonomous agents, decision-making is constantly happening during the process of repository browsing, task planning, programming, debugging, testing, and interactions with tools. As a result, an error in any step of reasoning can affect the entire chain of execution and lead to failure in task accomplishment.

Initial studies have mostly been concerned with addressing failures within the context of automated program repair. In such cases, the problem-solving involved is related to detecting faults within program code, proposing potential fixes for them and evaluating such fixes using existing test suites. According to recent surveys, it has been found that the use of Large Language Models has contributed to significant improvements in automated program repair by improving fault localization, the quality of proposed patches and overall accuracy of fixes in a wide range of programming-related problems [18]. However, such approaches tend to focus on treating faults as isolated source code issues and fail to take into consideration the reasoning process behind autonomous decision making.

In addition, the appearance of repository-aware programming agents has made the difficulty of execution errors even

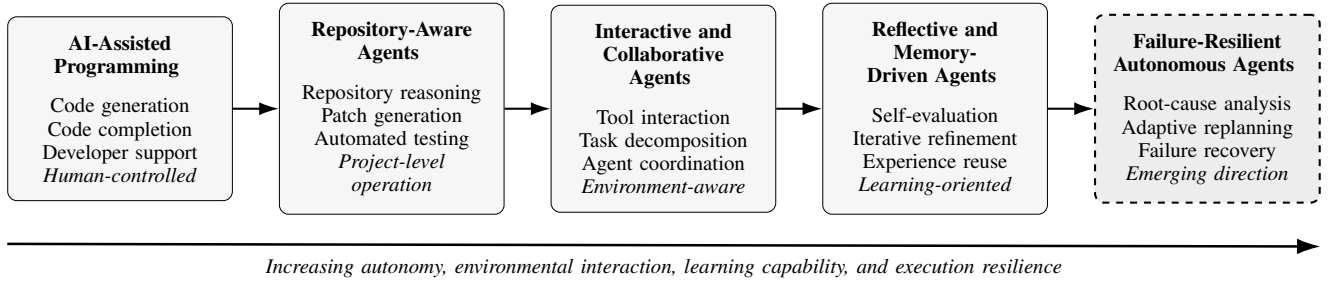


Fig. 2. Evolution of intelligent software engineering agents from AI-assisted programming toward reflective and failure-resilient autonomous systems. The reviewed literature indicates a gradual integration of repository-level reasoning, environmental interaction, multi-agent collaboration, self-reflection, memory-driven learning, and adaptive recovery capabilities. Failure-resilient autonomous software engineering remains an emerging research direction.

more apparent. Unlike previous repair mechanisms, repository-aware coding agents constantly interact with the software repository, with other development tools, with the command line, and testing framework. This interaction increases the number of possible errors, such as wrong task decomposition, invalid repository operation, wrong choice of tool, execution-time exceptions, dependency errors, as well as cascading errors in reasoning. As a result, the current benchmarking methods for coding agents changed their attention from evaluation of the correctness of the code to the evaluation of the whole software engineering process under realistic development conditions [19], [20]. Nevertheless, the main purpose of these benchmarks is to evaluate the success of the task, not the reason of the error.

Consequently, current research has started to examine execution failures as a unique research problem in itself rather than as mere failures in the execution phase. Through examining cases of autonomous agent failures, it has been found that execution failures are generally caused by reasoning flaws, planning problems, memory retrieval errors, interaction difficulties with the environment, and inadequate tool use, in addition to mere code generation problems [21]. The above observation implies that software engineering failures in autonomous agents are likely cumulative in nature, whereby one initial reasoning error cascades into a series of planning mistakes, ultimately leading to total failure of the task.

A further critical research direction is related to enhancing autonomous debugging via iterative execution-based reasoning. Modern debugging approaches use execution feedback, output from various tools, compiler messages, and other execution information to reason about new iterations. Instead of stopping at the point where execution fails, agents perform repeated analysis of the results obtained, change their implementation approach, and produce better solutions via successive iterations of execution. This type of debugging has proved more reliable than the traditional approach to software generation based only on one-time reasoning.

While iteration helps in achieving better execution, most of the recovery mechanisms currently available are reactive in nature. Most of the current systems do not conduct recovery until there is an actual failure; either they reattempt previous reasoning with slight changes or develop new implementa-

tions. The problem with such processes is that they usually lack specific methods for determining reasons for failures, as well as differentiating reasoning errors from execution errors and storing information on failures for future use. Thus, agents tend to make the same reasoning errors again and again.

Moreover, the emergence of benchmarking-based researches has drawn attention towards the necessity of developing more thorough approaches of measurement, which would be able not only to estimate the completion of the final task but also measure the autonomous recovery. Current benchmarks allow one to assess the performance of software engineering through such measures as task success rate, execution correctness, benchmark completion, and correctness of the task itself. On the other hand, not much work has been done on the estimation of the recovery capacity of an agent, which includes finding the reasons for failure, changing the reasoning and execution process, and recovering from the failed attempt at intermediate execution.

As an aggregate, the above-mentioned body of literature shows that significant advancements have been made in the direction of enhancement of autonomous debugging, execution monitoring, and software repair. However, failure recovery is still relatively less explored in comparison with reasoning, planning, and learning based on memory. The vast majority of existing literature focuses on reflection, debugging, planning, or software repair independently without trying to combine them into one single recovery process able to analyze execution traces, find the cause of the failure, use its past experience and make corresponding adjustments in reasoning in the future. Given the growing complexity of the development tasks undertaken by autonomous software engineering agents, overcoming such gaps will be crucial in the future.

E. Planning, Tool Utilization, Benchmarks, and Emerging Agent Architectures

Modern autonomous software engineering agents' capabilities go way beyond code generation and debugging. The success of such agents in practice largely hinges on the development of intelligent planning techniques, third-party development tools, standard benchmarks for evaluation, as well as the scalability of agent architecture that altogether make reliable autonomous software engineering possible. In turn,

such directions in research serve as an operational basis for efficient reasoning, reflection, memory, and failure recovery in software engineering.

Planning has become one of the crucial aspects of autonomous decision making since the software engineering processes cannot be considered as a chain of just a few reasoning steps. An autonomous agent needs to divide its high-level goals into a series of subtasks which include, but are not limited to, exploration of the repository, analyzing dependencies between components of the project, modifying the code, testing, debugging, and validation. Recent research proves that the hierarchical approach to planning results in significant increase of the efficiency of agent execution by allowing agents to break down complex processes in software engineering into executable actions, rather than inferring them directly from the language model [22], [23].

Very similar to the previous point is the rise in the usage of external software engineering tools. The contemporary autonomous agent not only relies only on its language model capabilities but also interacts with the integrated development environment, command line interface, compiler, debugger, testing framework, version control system, and application programming interface (API). These kinds of interaction provide much larger capabilities to the autonomous agent in terms of extracting external information, performing development-related commands, validating generated code, and receiving real execution feedback for future reasoning processes. All the extensive studies conducted on the use of tool-assisted language models prove that proper use of the tools greatly increases applicability of such software engineering agents [24], [25].

Moreover, the fast development of autonomous software engineering has led to an urgent need for standardized methods of evaluating autonomous agents' performance. The former approaches to evaluation focused on the code correctness and benchmark accuracy; nevertheless, today the autonomous agents need a much broader set of criteria for their evaluation. Modern benchmarking frameworks include such criteria as repository knowledge, reasoning, planning, tools, execution and task completion within the context of realistic software engineering environments [19], [20], [26], [27]. Today, the need for evaluation methodologies is crucial in order to compare the performance of autonomous agents in various software engineering scenarios using standardized data for experiment [19], [20], [26], [27]. However, the current approaches to evaluation still focus on outcomes and are rather ineffective in reasoning, adaptation and recovery during the execution process.

Not only individual reasoning components but also recent research efforts concentrate on the development of full-fledged autonomous agent architectures that would combine all of the reasoning abilities (reasoning itself, planning, memory, reflection, environment interaction) in one software engineering system. All existing surveys prove the idea that modern agent architectures become more and more complex and move from single agent pipelines to modular and collaborative architec-

tures consisting of several reasoning components dedicated to such functions as planning, memory management, tools execution, communication, and decision-making [28]–[31].

The next emerging trend is associated with the shift from the paradigm of isolated autonomous agents to continuously evolving software engineering ecosystems. Instead of executing individual programming tasks, agentic software engineering systems cooperate with external services, development infrastructure, organizational knowledge databases, and even other agents to perform complex software engineering tasks. This trend can be considered a part of the larger shift from developing individual language models in artificial intelligence to intelligent software engineering ecosystems that are able to learn, collaborate and make decisions continuously [29], [31], [32].

However, all things considered, the literature reviewed indicates that recent progress in the areas of planning methodologies, tool usage, benchmarks creation, and modular agent design significantly broadened the scope of autonomous software engineering tools capabilities. All these different technologies together create the necessary framework to facilitate the process of intelligent reasoning, self-reflection, memory-based learning, and autonomous operation within the realistic environment of software engineering. Nonetheless, despite these advancements in the research domain, current investigations continue to examine each one of these aspects independently, producing fragmented autonomous workflows that cannot ensure their robustness in the course of multi-step processes. In order to understand these advantages and disadvantages systematically, a comparative review of the literature reviewed is needed. Thus, the next chapter of this paper presents a comparative analysis of the chosen literature.

V. COMPARATIVE ANALYSIS

Even with advancements that have been made in automated software engineering through reasoning, planning, self-reflection, memory, debugging, tool usage and collaborative agent architectures, the studies considered here vary in terms of their goals, methods, environments for evaluation and capabilities.

These studies will be compared based on their focus, contributions, self-reflection, memory, planning, tool usage, recovery from failures, evaluation environments and weaknesses. E, P and “–” denote the capabilities that are addressed, partially addressed and not considered in each study, respectively, in Table III. Table III.

TABLE III: Comparative Analysis of the Selected Studies

Ref.	Primary Focus	Principal Contribution	Self-Ref.	Mem. Plan.	Tool	Rec.	Evaluation Context	Major Limitation
[1]	Multi-agent systems for software engineering	Considers LLM-based multi-agent systems throughout the software development life cycle and highlights coordination and architectural problems.	P	P	E	E	P Literature review and software engineering case studies	Includes wide coverage, but lacks empirical analysis of failure-recovery integration.
[2]	AI teammates and Software Engineering 3.0	Investigates how autonomous coding agents have become software engineering teammates in practice, based on evidence from large repositories and pull requests.	P	P	E	E	P Large repository and pull request datasets, developers, and coding agents	Emphasizes new development techniques, not explicit root cause identification and failure recovery.
[3]	Autonomous repository-level program improvement	Proposes AutoCodeRover for exploring repositories, locating failures, proposing patches, and validating them.	P	–	E	E	P SWE-Bench repository-level software problems	The proposed recovery strategy is still patch-based without reuse of failure knowledge between different tasks.
[4]	Interfaces between agents and computers for software engineering	Proposes SWE-agent and shows that well-designed interfaces enhance navigation, editing, testing, and debugging of the repository.	P	–	E	E	P SWE-Bench and HumanEvalFix	Depends heavily on interface design and lacks root-cause-oriented recovery.
[5]	Autonomous program repair using large language models	Proposes RepairAgent that autonomously localizes faults, generates patches, executes tools, and validates the repair iteratively.	P	–	E	E	E Automated program-repair tasks in practice	Focuses on fixing source-code problems rather than reasoning and planning issues.
[6]	Open platform for generalist software agents	Offers OpenHands, a platform for building, running, and evaluating autonomous software engineering agents.	P	P	E	E	P Repository-level software-engineering tasks	Offers infrastructure and evaluation framework without dedicated failure-recovery mechanisms.
[7]	Verbal Reinforcement Learning through Reflection	Introduces Reflexion and allows agents to produce verbal reinforcement feedback, store reflections, and enhance future attempts.	E	E	P	P	E Reasoning, decision-making, and programming tasks	The recovery process depends on the quality of agent reflections and does not include detailed diagnosis of execution failures.
[8]	Iterative Improvement through Self-Feedback	Introduces SELF-REFINE and separates generation, self-evaluation, and refinement without retraining an additional model.	E	–	P	–	P Text generation, reasoning, and programming tasks	Improves generated outputs through iterations but does not possess persistent memory or detailed execution-failure diagnosis.
[10]	Self-Debugging of Generated Programs	Allows LLMs to evaluate generated programs, understand execution feedback, and debug logical errors.	E	–	P	P	E Program generation and debugging benchmarks	Primarily measures individual programs rather than repository-based coding-agent trajectories.
[9]	Effect of self-reflection on problem solving	Empirically studies the role of reflective reasoning in enhancing problem-solving skills of LLM agents.	E	–	P	–	P General problem-solving experiments	Reflection may be ineffective on some tasks and may produce incorrect reasoning.
[11]	Experiential learning by LLM agents	Proposes ExpeL, in which agents gain, store, and use lessons learned from successful and unsuccessful experiences.	E	E	P	P	E Reasoning and agent tasks	Transfer of experience may become ineffective when tasks differ greatly from stored examples.
[12]	Reflective multi-agent cooperation	Proposes reflective cooperation in which agents specialize and exchange comments to improve intermediate solutions.	E	P	E	P	P Reasoning and multi-agent tasks	Reflection among multiple agents requires additional communication and coordination.
[13]	Review of LLM self-correction approaches	Groups self-correction approaches into feedback, refinement, verification, and iterative revision categories.	E	P	P	P	P Literature on cross-domain self-correction	Current approaches are fragmented, task-specific, and difficult to evaluate using a common methodology.
[14]	Self-improving generalist computer agents	Presents OS-Copilot for continual improvement through interaction, tool execution, and experience reuse.	E	E	E	E	P Operating-system environment for interactive use	Focuses on general computer interaction and provides limited specialization for repository-related code defects.
[18]	LLM automated program repair approach	Systematically reviews LLM-based approaches to fault localization, patch generation, validation, and repair.	P	P	P	E	E Automated program repair literature	Focuses on software faults and patch correctness rather than execution trajectories of autonomous agents.
[19]	Benchmark for software-engineering repositories	Proposes SWE-Bench using GitHub issues requiring repository knowledge, code modifications, and testing verification.	–	–	P	E	– GitHub issues from open-source projects	Tests final issue solving but provides limited information about intermediate failures and recovery.
[20]	Evaluation of LLM agents in general terms	Proposes AgentBench for reasoning, planning, decision-making, and interaction across various agent environments.	–	–	E	E	– Various environments for interactive agents	Provides broad evaluation but does not focus on software-specific root cause analysis and recovery.
[26]	Evaluation of autonomous agents based on agents	Proposes Agent-as-a-Judge for evaluating complex trajectories through other intelligent agents.	P	–	P	P	– Agent evaluation and trajectory judging tasks	Evaluation quality depends on the judging agent.

Continued on next page

Table III – Continued from previous page

Ref.	Primary Focus	Principal Contribution	Self-Ref.	Mem.	Plan.	Tool	Rec.	Evaluation	Context	Major Limitation
[33]	Self-improving coding agents	Describes SICA, an agentic programming tool capable of modifying itself to enhance efficiency and tool capabilities.	E	P	E	E	P	SWE-Bench, LiveCodeBench, and synthetic benchmarks		Globally optimizes agent implementation without diagnosing and recovering from individual task execution failures.
[34]	Index of deployed agentic AI systems	Surveys 30 deployed agents by technical capability, autonomy, environment interaction, evaluation, safety, and transparency.	P	P	P	E	–	Documentation and developer information for 30 deployed agents		Uses mainly publicly available information without experimental evaluation.
[15]	Dynamic agentic memory organization	Proposes A-MEM to establish dynamic associations, organization, evolution, and retrieval of agent experiences.	P	E	P	–	P	Long-term agent memory and reasoning		Focuses on memory organization rather than failure diagnosis or recovery of coding agents.
[17]	Long-term conversational memory assessment	Proposes LOCOMO for evaluating long-term memory, temporal reasoning, and consistency in prolonged conversations.	–	E	–	–	–	Long-term conversational memory benchmark		Evaluates conversational memory but does not consider software engineering agents.
[16]	Memory architecture for multi-agents	Introduces MIRIX with episodic, semantic, procedural, resource, and knowledge memories.	P	E	P	P	P	Long-term multimodal and agent memory		Memory coordination can be complex and introduce additional storage and computational overhead.
[32]	Vibe coding versus agentic coding	Differentiates prompt-based code generation from autonomous, tool-driven, and goal-oriented code generation.	P	P	E	E	P	Conceptual and practical analysis		Provides conceptual insights but lacks extensive empirical evaluation.
[27]	Benchmarking the safety of code agents	Introduces RedCode-Exec and RedCode-Gen frameworks for assessing code execution risk and malicious code generation.	–	–	P	E	–	Code-execution and code-generation safety benchmarks		Assesses safety but does not address post-failure recovery.
[21]	Failure taxonomy, benchmark, and debugging for agents	Introduces AgentErrorTaxonomy, AgentErrorBench, and AgentDebug for failure classification, root cause analysis, and iterative debugging.	E	E	E	E	E	General LLM-agent failures and iterative debugging tasks		The general agent framework requires further validation in coding contexts.
[24]	Benchmarking for augmented LLMs	Introduces API-Bank to test tool calling, tool retrieval, tool-call planning, and executable tool use.	–	–	E	E	P	API Call, Retrieve-and-Call, and Plan-Retrieve-and-Call tasks		Provides limited evaluation of persistent learning, root cause identification, and long-term recovery.
[25]	Survey on tool use by LLMs	Surveys architectural approaches for discovering, selecting, calling, and learning to use external tools.	P	P	E	E	P	Survey of tool-augmented LLM methodologies		Broad survey without empirical comparison of software-engineering recovery techniques.
[22]	Task planning and tool usage for AI agents	Presents TPTU covering perception, task planning, tool use, learning, reflection, memory, and summarization.	P	P	E	E	P	One-step and sequential agent environments		Provides a conceptual framework but lacks evaluation in programming environments.
[23]	Low-shot grounded planning and replanning	Introduces LLM-Planner for high-level planning and dynamically grounded replanning in embodied tasks.	–	P	E	P	E	ALFRED embodied-task environment		Demonstrates replanning outside software engineering and lacks repository-level coding evaluation.
[28]	Survey of recent agent architectures	Studies single-agent, multi-agent, vertical, and horizontal architectures for reasoning, planning, memory, reflection, and tool calling.	E	E	E	E	P	Literature synthesis at architecture level		Provides extensive architectural discussion but limited empirical analysis of failure recovery.
[35]	Multi-agent robotic task planning	Proposes SMART-LLM for task decomposition, coalition formation, resource allocation, and completion by heterogeneous robots.	–	–	E	P	P	AI2-THOR simulation and physical robots		Robotic-domain assumptions restrict applicability to repository-level software-engineering failure scenarios.
[29]	Agentic AI concepts	Reviews frameworks, architectures, applications, evaluation approaches, and challenges in agentic AI research.	E	E	E	E	P	Systematic review of 143 studies		Its broad scope makes an in-depth study of self-reflective recovery in autonomous coding systems difficult.
[30]	Comprehensive review of agentic AI	Provides a PRISMA-based taxonomy of symbolic and neural architectures, applications, frameworks, governance, and future perspectives.	E	E	E	E	P	Systematic review of 90 studies		Focuses on agent architectures and governance rather than coding-specific execution recovery.
[31]	LLM-based autonomous agents	Presents a unified architecture covering profiling, memory, planning, action, applications, and challenges in agent evaluation.	E	E	E	E	P	Autonomous-agent literature		Provides a general taxonomy without a specialized recovery architecture based on root cause analysis.

Notation: E = explicitly implemented or centrally evaluated; P = partially implemented, indirectly supported, or discussed; – = not addressed as a principal capability. Self-Ref. = self-reflection, Mem. = memory, Plan. = planning, Tool = tool utilization, Rec. = failure recovery.

A. Cross-Study Capability Analysis

Table III reveals that planning and tool use are the most common capabilities found in the literature surveyed. Repository-based agents, autonomous agents, and tool-assisted language models typically involve task decomposition, command execution, environment interaction, and validation. With these capabilities, autonomous systems have progressed from generating isolated pieces of code to executing complete software-engineering workflows.

Self-reflection is becoming more common, especially in research focused on iterative improvement, verbal feedback, experiential learning, and self-debugging. Reflection is typically used as a local post-execution correction strategy rather than as a continuous capability able to monitor the whole execution process. While reflection usually improves the current output or reasoning attempt, it does not necessarily reveal the source of the problem.

Research into memory provides mechanisms for saving, organizing, and retrieving previous experience. Such mechanisms can enhance consistency over time and minimize repeated reasoning efforts. However, memory research is rarely integrated directly with failure detection and root-cause analysis. Therefore, agents may retrieve previous experiences without establishing whether those experiences are valid, relevant, or appropriate in the current software-engineering context.

The ability to recover from failures has been used the least consistently. Although various systems provide failure-recovery capabilities through repair systems, reflective agents, and debugging mechanisms, very few studies combine failure detection, root-cause analysis, memory retention, replanning, feedback, and re-execution within a unified framework.

B. Critical Discussion

The comparison suggests three major categories of research. The first category includes repository-level coding agents and automated program repair approaches, which offer practical interaction with source-code repositories, software-development tools, test sets, and execution environments. While these studies demonstrate strong software-engineering competence, they focus mainly on successful task accomplishment or successful execution of patches rather than learning through failed execution paths.

The second category includes approaches for self-reflection, self-correction, and memory. These studies substantially enhance iterative reasoning, experience storage, and adaptation processes. However, many are evaluated on generic reasoning tasks, conversational interactions, embodied tasks, and isolated coding tasks rather than real-world repository-level software-engineering tasks. Their performance during cascading failures and long-term interaction with tools remains poorly understood.

The third category involves benchmark sets, architectural descriptions, and general agent-evaluation frameworks. These studies provide useful taxonomies, evaluation environments, and architectural descriptions. However, most benchmark sets

are outcome-driven and evaluate task success, functional correctness, or correct tool usage. Metrics for failure localization, recovery efficiency, prevention of repeated failures, recovery cost, and improvement across successive attempts remain uncommon.

In summary, the review highlights substantial knowledge on individual elements of autonomous failure recovery, including reflection, memory, planning, debugging, tool use, and execution feedback. However, these elements are frequently investigated separately, limiting their integration within autonomous coding agents. This fragmentation hinders the ability of coding agents to diagnose failures, learn from previous executions, modify unsuccessful plans, and recover from complex repository-level tasks.

C. Future Research Directions

The research deficiencies discussed in the above-mentioned sub-section prove the following point: Autonomous software engineering is evolving from isolated intelligent components to integrated, adaptive, and continuously learning software engineering system. Accordingly, future research should be aimed not only at improvement of single components but also at creating of autonomous systems able to perform robust long-horizon software engineering in realistic software development environments.

One possible way of future research is related to creation of integrated autonomous recovery frameworks consisting of reasoning, planning, self-reflection, memory management, debugging, and adaptive execution. In contrast to the mentioned independent components, future systems should provide interaction between them throughout the entire process of software engineering. Such integration will allow for autonomous agent's monitoring of execution process, interpretation of failures, updating of reasoning policies, retrieval of experience, and adjustment of decisions without the use of isolated recovery techniques.

An additional direction worth exploring is root cause analysis and its implementation into autonomous coding agents. Currently used approaches tend to fix the problem of execution without discovering how exactly the reasoning process led to this failure. In the future, there is a need to develop approaches capable of tracing the process of execution failures throughout several reasoning phases, separating the problem into planning mistakes and execution flaws, finding connections between navigating the repository, using tools, reasoning, and the execution process outcome. This approach can greatly increase the efficiency of autonomous systems working in the domain of software engineering.

In addition, the memory architecture needs to go beyond just storing experiences and move towards contextual adaptation and dynamic learning. The system needs not only to recall previously executed code but to consider whether the previously collected data is relevant and applicable to the current software engineering task. There are three promising directions in memory architecture development – dynamic

memory prioritization, experience abstraction, and failure-aware memory.

Moreover, it is necessary to emphasize the demand for creating more comprehensive benchmark datasets and evaluation methods for autonomous recovery purposes. Any future benchmarking frameworks have to go beyond evaluating task execution and incorporate numerical metrics of recovery performance, including recovery accuracy, cause identification, adaptation, planning, execution efficiency, error avoidance, computational cost, and learning. In such a way, the development of standardized recovery-oriented benchmarks would allow for the objective evaluation of autonomous coding agents and accelerate the development of autonomous software engineering systems.

One more direction of further development of the considered technology concerns the improvement of computational efficiency of the proposed solutions. Reflection, debugging, and execution multiple times take much time and require additional computation. Therefore, further studies should be devoted to creating adaptive recovery techniques that could balance between execution robustness and computational efficiency, providing reflection optimization, recovery triggering, planning depth, and resource-efficient reasoning. Such approaches could help in efficient utilization of autonomous software engineering agents in industrial settings.

Lastly, next-generation autonomous software engineering systems must shift focus from reactive execution recovery to proactive execution management. In addition to correcting errors during execution, intelligent agents need to monitor execution paths, predict failure probability, detect hazardous reasoning patterns, and plan ahead for possible issues that may lead to errors propagating across the entire software engineering process. Predictive failure detection, uncertainty-based planning, and continuous execution monitoring all constitute a significant advancement towards reliable autonomous software engineering processes with minimum human intervention.

In general, future research efforts need to concentrate on the incorporation of reasoning, memory, planning, execution monitoring, and adaptive execution recovery into autonomous software engineering systems. By developing these research directions, not only can the robustness and efficiency of intelligent coding agents be improved, but also intelligent software engineering systems capable of handling complicated software engineering tasks can be developed faster. The integrated autonomous failure recovery workflow synthesized from the reviewed literature is illustrated in Fig. 3.

VI. CONCLUSION

Substantial progress was achieved in the field of autonomous software engineering by creating LLM-powered coding agents that can comprehend repositories, generate code, debug, plan, utilize tools, and execute tasks iteratively. The review focused on the latest studies related to self-reflection, memory, planning, multi-agents cooperation, benchmarking, and failure recovery to summarize the state-of-the-art knowledge on autonomous coding systems.

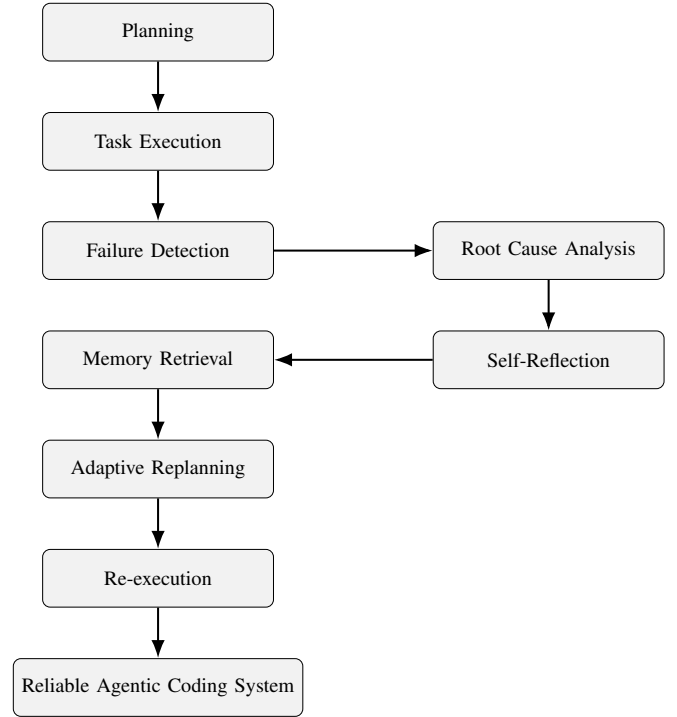


Fig. 3. Workflow for integrated autonomous failure recovery based on the reviewed literature. The diagram depicts how planning, failure identification, cause identification, reflection, memory recall, adaptation, and re-execution can be integrated to make agentic AI coding more reliable.

It has been demonstrated that self-reflection can be used to enable iterative reasoning and correction, while memory can help reuse previous experiences. Planning and tool utilization allow for the accomplishment of multi-step tasks, and multi-agents cooperation might lead to better performance on challenging software engineering tasks. Nevertheless, all of these features are studied independently. The review identified a gap related to the lack of combined failure recovery solutions that integrate reflection, cause analysis, memory, planning, and adaptive execution. There is a lack of approaches for the assessment of the effectiveness of the failure recovery process, its efficiency, root cause detection, and proactive failure prevention.

With respect to these results, future studies need to concentrate on integrated failure recovery architectures, cause aware reasoning, adaptive memory, recovery oriented benchmarks, efficient resource utilization and predictive failures. In general, this review is an organized summary of the studies conducted on agentic programming focusing on failure recovery and self reflection capabilities along with limitations and research directions.

REFERENCES

- [1] J. He, C. Treude, and D. Lo, "Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 124:1–124:30, 2025.

- [2] H. Li, H. Zhang, and A. E. Hassan, "The rise of ai teammates in software engineering (se 3.0): How autonomous coding agents are reshaping software engineering," 2025.
- [3] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "Autocoderover: Autonomous program improvement," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2024, pp. 1592–1604.
- [4] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024. [Online]. Available: <https://openreview.net/forum?id=mXpq6ut8J3>
- [5] I. Bouzenia, P. Devanbu, and M. Pradel, "Repairagent: An autonomous, llm-based agent for program repair," in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025, pp. 2188–2200.
- [6] R. Brennan, G. Neubig, S. Peng *et al.*, "Openhands: An open platform for ai software developers as generalist agents," *arXiv preprint*, vol. arXiv:2407.16741, 2024.
- [7] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [8] A. Madaan, N. Tandon, P. Gupta *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [9] M. Renze and E. Guven, "Self-reflection in llm agents: Effects on problem-solving performance," in *First Workshop on Foundational Large Language Models (FLLM)*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.06682>
- [10] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: <https://arxiv.org/abs/2304.05128>
- [11] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "Expel: Llm agents are experiential learners," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, vol. 38, no. 17, 2024, pp. 19 632–19 640.
- [12] X. Bo, Z. Zhang, Q. Dai, X. Feng, L. Wang, R. Li, X. Chen, and J.-R. Wen, "Reflective multi-agent collaboration based on large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.
- [13] L. Pan, M. Saxon, W. Xu, D. Nathani, X. Wang, and W. Y. Wang, "Automatically correcting large language models: Surveying the landscape of diverse self-correction strategies," *arXiv preprint*, vol. arXiv:2308.03188, 2023.
- [14] Z. Wu, C. Han, Z. Ding, Z. Weng, Z. Liu, S. Yao, T. Yu, and L. Kong, "Os-copilot: Towards generalist computer agents with self-improvement," *arXiv preprint*, vol. arXiv:2402.07456, 2024.
- [15] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-mem: Agentic memory for llm agents," in *Proceedings of the 39th Conference on Neural Information Processing Systems (NeurIPS)*, 2025.
- [16] Y. Wang and X. Chen, "Mirix: Multi-agent memory system for llm-based agents," *arXiv preprint*, vol. arXiv:2507.07957, 2025.
- [17] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of llm agents," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 13 851–13 870.
- [18] Q. Zhang, C. Fang, Y. Xie, Y. Ma, W. Sun, Y. Yang, and Z. Chen, "A systematic literature review on large language models for automated program repair," *ACM Transactions on Software Engineering and Methodology*, 2026.
- [19] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "Swe-bench: Can language models resolve real-world github issues?" in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [20] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "Agentbench: Evaluating llms as agents," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [21] K. Zhu, Z. Liu, B. Li, M. Tian, Y. Yang, J. Zhang, P. Han, Q. Xie, F. Cui, W. Zhang, X. Ma, X. Yu, G. Ramesh, J. Wu, Z. Liu, P. Lu, J. Zou, and J. You, "Where llm agents fail and how they can learn from failures," *arXiv preprint*, vol. arXiv:2509.25370, 2025.
- [22] J. Ruan, Y. Chen, B. Zhang, Z. Xu, T. Bao, G. Du, S. Shi, H. Mao, Z. Li, X. Zeng, and R. Zhao, "Tptu: Task planning and tool usage of large language model-based ai agents," in *Foundation Models for Decision Making Workshop at NeurIPS*, 2023.
- [23] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, and Y. Su, "Llm-planner: Few-shot grounded planning for embodied agents with large language models," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 2998–3007.
- [24] M. Li, Y. Zhao, B. Yu, F. Song, H. Li, H. Yu, Z. Li, F. Huang, and Y. Li, "Api-bank: A comprehensive benchmark for tool-augmented llms," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 3102–3116.
- [25] Z. Shen, "Llm with tools: A survey," *arXiv preprint*, vol. arXiv:2409.18807, 2024.
- [26] M. Zhuge, C. Zhao, D. R. Ashley, W. Wang, D. Khizbullin, Y. Xiong, Z. Liu, E. Chang, R. Krishnamoorthi, Y. Tian, Y. Shi, V. Chandra, and J. Schmidhuber, "Agent-as-a-judge: Evaluate agents with agents," *arXiv preprint*, vol. arXiv:2410.10934, 2024.
- [27] C. Guo, X. Liu, C. Xie, A. Zhou, Y. Zeng, Z. Lin, D. Song, and B. Li, "Redcode: Risky code execution and generation benchmark for code agents," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.
- [28] T. Masterman, S. Besen, M. Sawtell, and A. Chao, "The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey," *arXiv preprint*, vol. arXiv:2404.11584, 2024.
- [29] A. Bandi, B. Kongari, R. Naguru, S. Pasnoor, and S. V. Vilipala, "The rise of agentic ai: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges," *Future Internet*, vol. 17, no. 9, p. 404, 2025.
- [30] M. A. Ali, F. Dornaika, and J. Charafeddine, "Agentic ai: A comprehensive survey of architectures, applications, and future directions," *Artificial Intelligence Review*, vol. 59, p. 11, 2026.
- [31] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
- [32] R. Sapkota, K. I. Roumeliotis, and M. Karkee, "Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic ai," *arXiv preprint*, vol. arXiv:2505.19443, 2025.
- [33] M. Robeyns, M. Szummer, and L. Aitchison, "A self-improving coding agent," *arXiv preprint*, vol. arXiv:2504.15228, 2025. [Online]. Available: <https://arxiv.org/abs/2504.15228>
- [34] L. Stauffer, K. Feng, K. Wei, L. Bailey, Y. Duan, M. Yang, A. P. Ozisik, S. Casper, and N. Kolt, "The 2025 ai agent index: Documenting technical and safety features of deployed agentic ai systems," in *Proceedings of the 2026 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. ACM, 2026, pp. 1536–1576.
- [35] S. S. Kannan, V. L. N. Venkatesh, and B.-C. Min, "Smart-llm: Smart multi-agent robot task planning using large language models," *arXiv preprint*, vol. arXiv:2309.10062, 2024.